

Szczegółowe Sprawozdanie z Projektu Nr 1

Wielowątkowość wysokopoziomowa w języku Java

Analiza porównawcza technik klasycznych i wysokopoziomowych

3 maja 2026

Spis treści

1	Cel i Zakres Projektu	2
2	Weryfikacja Zgodności z Poleceniem	2
3	Szczegóły Implementacji	2
3.1	Logika Zadania (KrokAnimacji)	2
3.2	Pula Wątków (ExecutorService)	2
4	Analiza Profilerem (VisualVM)	3
4.1	Zużycie Procesora (CPU Usage)	3
4.2	Stany Wątków (Thread States)	3
5	Wnioski Końcowe	3

1. Cel i Zakres Projektu

Głównym celem projektu było przeprowadzenie dogłębnej analizy porównawczej mechanizmów współbieżności w języku Java. Zbadano:

- Podejście klasyczne: Bezpośrednie użycie interfejsu `Runnable` oraz klasy `Thread`.
- Podejście wysokopoziomowe: Wykorzystanie interfejsów `Executor`, `ExecutorService` oraz `ScheduledExecutorService` z pakietu `java.util.concurrent`.

2. Weryfikacja Zgodności z Poleceniem

Analiza przykładowego rozwiązania wykazuje pełną zgodność z wymaganiami określonymi w pliku `polecenie.txt`:

1. **Zadanie programistyczne:** Zaimplementowano animację 100 elementów graficznych poruszających się po panelu Swing. Dodatkowo, każdy wątek wykonuje intensywne obliczenia matematyczne (`heavyCalculations`), co pozwala na realne obciążenie CPU.
2. **Implementacja klasyczna:** Metoda `startThreads()` tworzy listę obiektów `Thread`, z których każdy zarządza jednym zadaniem.
3. **Implementacja wysokopoziomowa:** Metody `startExecutorService()` oraz `startScheduledExecutorService()` wykorzystują pule wątków (`ThreadPoolExecutor`) o stałym rozmiarze (domyślnie 4).
4. **Analiza profilerem:** Przeprowadzono pomiary z wykorzystaniem narzędzia VisualVM 2.2.1 oraz narzędzi systemowych JDK (`jstack`, `jstat`).

3. Szczegóły Implementacji

3.1 Logika Zadania (KrokAnimacji)

Każde zadanie wykonuje pętlę, w której zamiast pasywnego oczekiwania (`sleep`), wywoływana jest funkcja obciążająca procesor:

```
1 public static void heavyCalculations() {
2     double wynik = 0;
3     for (int i = 0; i < 50000; i++) {
4         wynik += Math.sin(i) * Math.tan(i);
5     }
6 }
```

To podejście pozwala na badanie stanów wątków w warunkach wysokiej rywalizacji o zasoby.

3.2 Pula Wątków (ExecutorService)

W odróżnieniu od metody klasycznej, gdzie powstaje 100 wątków, w metodzie wysokopoziomowej zadania są kolejgowane do puli 4 wątków.

```
1 executorService = Executors.newFixedThreadPool(ThreadingProject_P1.pula);
2 for (RuchomyElement el : panel.getElements()) {
3     executorService.submit(new KrokAnimacji(el, panel, this));
4 }
```

4. Analiza Profilerem (VisualVM)

4.1 Zużycie Procesora (CPU Usage)

- **Metoda Klasyczna:** Gwałtowny wzrost obciążenia CPU. System operacyjny musi zarządzać przełączaniem kontekstu dla 100 wątków konkurujących o rdzenie procesora. Widoczny narzut na zarządzanie wątkami.
- **ExecutorService (FixedThreadPool):** Obciążenie stabilizuje się na poziomie odpowiadającym liczbie rdzeni/wątków w puli. Praca jest wykonywana płynnie, bez nadmiernego przełączania kontekstu.

4.2 Stany Wątków (Thread States)

Analiza zrzutu wątków (jstack) wykazała:

Cecha	Klasycznie	Wysokopoziomowo
Liczba wątków robotników	100	4 (stała)
Stan podczas pracy	RUNNABLE	RUNNABLE
Narzut pamięciowy	Wysoki (stack dla każdego wątku)	Niski (stała liczba stosów)
Zarządzanie kończeniem	Ręczne (interrupty w pętli)	Uproszczone (<code>shutdownNow()</code>)

Tabela 1: Porównanie charakterystyki wątków

5. Wnioski Końcowe

1. **Wydajność:** Pule wątków są znacznie bardziej efektywne przy dużej liczbie drobnych zadań. Zapobiegają one zjawisku *thread starvation* oraz nadmiernemu zużyciu pamięci RAM. 2. **Skalowalność:** API `java.util.concurrent` pozwala na łatwą zmianę strategii wykonywania zadań bez modyfikacji logiki samego zadania. 3. **Użyteczność:** `ScheduledExecutorService` jest jedynym rozsądnym wyborem dla zadań cyklicznych, zastępując błędne pętle z `Thread.sleep()`.